

Alta de departamentos en el servidor PagoSur

Sistemas Operativos · TP Obligatorio · Propuesta (versión 2, reformulada)

La idea

Una máquina Red Hat es el servidor central de **PagoSur**, una fintech ficticia. Funciona como repositorio de archivos de la empresa: cada departamento tiene su carpeta, sus usuarios y su espacio. Cuando se incorpora un departamento nuevo, el administrador ejecuta un script de *alta* que crea el grupo y los usuarios, la carpeta con cuota de disco, los permisos por rol, y las reglas que evitan que un proceso de esos usuarios tumbé el servidor. Todo queda registrado en un log de auditoría.

```
sudo ./asignar.sh --depto finanzas --usuarios finanzas.csv
```

Todo departamento recibe lo mismo, definido como constantes en el script: **256 MB de disco** y hasta **1 GB de memoria virtual por proceso**. Cuando el departamento se disuelve, `revocar.sh` deshace todo en orden inverso.

El servidor es el escenario común, pero la consigna exige 6 ejercicios de unidades distintas. Por eso cada pieza del alta es un ejercicio independiente, con su propia situación, su solución con capturas y su apartado teórico.

Qué cambió respecto de la propuesta original y por qué

La propuesta original repartía CPU y RAM por departamento con *slices* de systemd (cgroups). Al revisarla contra las consignas y el programa de la materia se decidió simplificar:

- **La consigna no exige ninguna herramienta concreta.** Exige 6 ejercicios de unidades distintas, consola de Red Hat, scripting en al menos 3, y que **cualquiera de los 4 integrantes pueda defender cualquier ejercicio** con la teoría de su unidad (puntos 6 y 16).
- **cgroups no está en el programa.** Y un slice por departamento no captura automáticamente los procesos de un usuario que se loguea: había que lanzarlos a mano con `systemd-run`. Difícil de explicar y fácil de que falle en la defensa.
- **El límite de memoria ahora se aplica con `limits.conf`**, que PAM activa en el login de cada usuario del grupo. Una línea por departamento, sin nada que lanzar. Es material clásico de la unidad de memoria.
- **El único recurso que se reparte de verdad por departamento es el disco**, con una cuota de proyecto de XFS sobre la carpeta. La memoria es una regla *por proceso* de los usuarios del grupo, pensada para proteger al servidor. Se explica así en el informe.
- El ejercicio de planificación de CPU (nice por grupo) se elimina: los ejercicios se numeran según las unidades de la materia, con Memoria como unidad 3 y Almacenamiento como unidad 4.
- El monitor de procesos pasa de C a bash (la propuesta ya decía "C o script"), y SELinux se reduce a una demostración en vez de configuración en el script.

Los 6 ejercicios

Ejercicio	Situación y qué hace	Herramientas Red Hat	Código	Teoría
1. Estructura del SO	Antes de dar de alta un departamento, el admin quiere saber en qué estado está el servidor. El inventario lee <code>/proc</code> y reporta CPUs y carga (¿hay contención?), RAM disponible (¿cuántos procesos de 1 GB entran?) y discos con su espacio libre (¿cuántos discos simulados del ej. 4 entran?).	<code>/proc/cpuinfo</code> , <code>/proc/loadavg</code> , <code>/proc/meminfo</code> , <code>/proc/partitions</code> , <code>statvfs</code> , <code>strace</code> , <code>lscpu</code> , <code>free</code> , <code>df</code>	C	Syscalls, modo usuario / kernel, <code>/proc</code>
2. Procesos	Un usuario deja un proceso colgado consumiendo CPU y se va. El monitor recorre los procesos de los usuarios del departamento, baja la prioridad al que supera el umbral y, si reincide, lo termina. Reporta zombies.	<code>ps</code> , <code>top</code> , <code>pstree</code> , <code>renice</code> , <code>kill</code>	Script	Estados, PCB, señales, zombies
3. Memoria	Un proceso pide memoria sin parar hasta que el servidor swapea. Ningún proceso de un usuario del departamento puede pedir más de 1 GB: <code>malloc</code> falla solo en ese proceso y el servidor sigue normal.	<code>limits.conf</code> , <code>ulimit</code> <code>-v</code> , <code>free</code> , <code>vmstat</code> , <code>pmap</code>	Script + C (<code>malloc</code> en loop)	Memoria virtual, paginación, swap, OOM killer
4. Almacenamiento	Cada departamento necesita su carpeta con un tope de 256 MB que no pueda superar, y que se pueda agrandar a demanda. Volumen lógico XFS sobre un loop device, montaje persistente y cuota de proyecto. <code>lvextend</code> a mano en la demo.	<code>losetup</code> , <code>pvcreate</code> , <code>vgcreate</code> , <code>lvcreate</code> , <code>mkfs.xfs</code> , <code>fstab</code> , <code>xfs_quota</code> , <code>lvextend</code>	Script	Sistemas de archivos, inodos, journaling, LVM
5. (a definir)	Unidad 5 de la materia: ejercicio pendiente de elegir.			
6. Seguridad	Alta de usuarios desde un CSV con rol (jefe, analista, auditor): grupo del departamento, carpeta con SGID, auditor de solo lectura por ACL, sudo limitado para el jefe, clave que vence. SELinux como ejemplo de MAC en la demo.	<code>useradd</code> , <code>groupadd</code> , <code>chage</code> , <code>chmod</code> , SGID, <code>setfacl</code> , <code>sudoers</code> , <code>ls</code> <code>-Z</code> , <code>restorecon</code>	Script	DAC vs MAC, mínimo privilegio

Cómo funciona de punta a punta

1. Alta (el admin, una vez por departamento)

```
sudo ./asignar.sh --depto finanzas --usuarios finanzas.csv
```

El orquestador valida los dos parámetros, muestra el estado del servidor y llama a los módulos en este orden, porque cada uno depende del anterior:

#	Módulo	Qué deja en el sistema
1	01_inventario	Imprime CPUs y carga, RAM disponible, discos y espacio libre en /. No cambia nada.
6	06_seguridad.sh	Grupo finanzas. Usuarios del CSV con clave a cambiar en el primer login y vencimiento a 90 días. /etc/sudoers.d/finanzas para el jefe.
4	04_almacenamiento.sh	LV /dev/vg_pagosur/lv_finanzas de 256 MB en XFS, montado en /srv/finanzas vía fstab con pquota. Carpeta root:finanzas modo 2770. ACL de solo lectura para el auditor. Cuota de proyecto de 256 MB.
3	03_memoria.sh	/etc/security/limits.d/depto-finanzas-memoria.conf con @finanzas hard as 1048576.

Cada módulo escribe una línea en logs/asignador.log con fecha, usuario y acción.

2. Uso (los usuarios del departamento, todos los días)

No hay nada que "acceder" ni configurar del lado del usuario. Entra con ssh o su - ana y en ese login PAM le aplica los límites del grupo:

```
su - ana                                # analista de finanzas
ulimit -v                              # -> 1048576 (ej. 3: máx. 1 GB/proceso)
cd /srv/finanzas && cp ~/informe.pdf .   # su carpeta; hereda el grupo por SGID
su - auditor1 -c "touch /srv/finanzas/x" # falla: el auditor solo lee (ACL, ej. 6)
dd if=/dev/zero of=/srv/finanzas/relleno bs=1M # corta en 256 MB (cuota, ej. 4)
```

3. Vigilancia (ej. 2, en su propia terminal)

```
sudo ./modulos/02_monitor.sh finanzas 50 # umbral 50 % de CPU
# en otra terminal, como ana: yes > /dev/null (satura un núcleo)
# el monitor lo detecta, le hace renice, y si sigue, kill -TERM / kill -KILL
```

4. Baja (cuando el departamento se disuelve)

```
sudo ./revocar.sh --depto finanzas
```

Pide confirmación y deshace todo en orden inverso: borra el archivo de limits.d, desmonta y elimina el volumen y su entrada en fstab, y por último elimina usuarios, grupo y sudoers. Sirve para repetir la demo desde cero.

Arquitectura

```
src/
├── asignar.sh          # alta: valida parámetros y llama a los módulos
├── revocar.sh         # baja: deshace asignar.sh en orden inverso
├── lib/log.sh          # log() compartida -> logs/asignador.log
└── módulos/
    ├── 01_inventario.c # Ej. 1 – estado del servidor (/proc, syscalls)
    ├── 02_monitor.sh   # Ej. 2 – control de procesos (ps / renice / kill)
    ├── 03_memoria.sh    # Ej. 3 – memoria máxima por proceso vía limits.d
    ├── 03_carga_ram.c   # Ej. 3 – malloc en loop hasta que falla
    ├── 04_almacenamiento.sh # Ej. 4 – loop device + LVM + XFS + cuota de proyecto
    └── 06_seguridad.sh  # Ej. 6 – usuarios, permisos, ACL, sudo
```

El orquestador es corto y no contiene lógica de los ejercicios. Cada módulo vive en su propio archivo, se ejecuta solo, y tiene en el encabezado la situación, las herramientas, la demo y una lista de pasos. Así cada integrante trabaja en el suyo y cualquier pieza se puede mostrar y defender por separado.

Cumplimiento de la consigna

Requisito	Cómo se cumple
6 ejercicios, uno por unidad (punto 4)	Cada módulo es un ejercicio con su unidad.
Consola de Red Hat en todos (punto 10)	Todos se resuelven y demuestran por terminal.
Shell scripting en al menos 3 (punto 14)	Ejercicios 2, 3, 4, 6 y el orquestador.
C/C++, valorado (punto 15)	Ejercicio 1 (inventario) y la carga de prueba del 3.
Enunciado hipotético creativo (puntos 8 y 13)	Escenario de empresa único; cada ejercicio arranca con una situación de una oración.
Apartado teórico por ejercicio (punto 11)	Columna "Teoría" de la tabla como punto de partida.
Cualquiera defiende cualquier ejercicio (puntos 6 y 16)	Módulos mínimos, sin herramientas fuera del programa. Cada informe cierra con preguntas de la unidad y sus respuestas, tomadas de los cuestionarios.

Consideraciones técnicas

- **Discos:** siempre un archivo disperso montado como loop device (`truncate -s 1G` + `losetup`), nunca discos reales. Declara 1 GB pero ocupa solo lo escrito. Snapshot de la VM antes de tocar LVM.
- **Tamaños a escala de demo:** la VM del lab tiene ~2 GB libres en `/`, así que el disco simulado es de 1 GB y cada departamento recibe 256 MB. La demo es la misma que con cientos de GB (la cuota corta, el VG se agota en el cuarto departamento) y el `dd` termina en segundos. En producción se cambia la constante `DISCO` del script.
- **Cuota de proyecto en XFS:** el volumen debe montarse con `pquota`; si no, `xfs_quota` no tiene efecto.
- **limits.conf:** los límites se aplican en el login (PAM). Para probarlos hay que entrar con `su - usuario` (con guion) o `ssh`; un `su usuario` sin guion no los carga.
- **Límite por proceso, no por departamento:** el 1 GB aplica a cada proceso de cada usuario del grupo. Un usuario con diez procesos usa diez veces eso. Es una limitación conocida y se explica en el informe; la cuota de disco sí es por departamento.

- **Entorno del lab:** verificar si se resetea entre sesiones. Todo vive en el repositorio para reconstruir rápido.
- **Independencia:** cada ejercicio debe poder demostrarse solo; un error en uno no rompe la demo de los demás.